Using macros for generic structures in C

Back when I was a moderately novice C programmer, I was looking at some project [1] and saw that someone had included the same file twice. “That’s strange,” I said to myself, “I wonder why anyone would do that.” After doing some poking, I discovered that the included file behaved differently depending on values that could be defined or not defined. So the programmer defined some values, included the file for one effect, then defined other values, and included the file for another effect.

At the time, that approach struck me as a stupid and annoying hack.
As I’ve become a more experienced programmer, I’ve realized that there can be reasonable reasons to make such a design choice. It can even help with DRY code. Let’s consider an example.

Good programmers build or use data structures [2]. In some languages, like Java or Scheme, you can build a generic data structure, and use it for various types [3]. That’s harder in C. Some programmers use pointers for the values in their data structures. When they want to store strings, they can store pointers to strings, or even just the strings themselves. When they want to store `People` structs, they can store pointers to those structs. And so on and so forth. That’s not particularly elegant, and it seems to work less well if you want to store primitive types. Storing pointers also seems to be a step on a dangerous path toward memory errors [10].

Is there an alternative? It turns out that modern C macros provide a somewhat reasonable alternative. Let’s see how.

Let’s start by defining a simple data structure. Let’s implement a stack, a commonly used, but simple, data structure. C likes us to specify the type we’re storing, we’ll start by designing a stack of integers.

Think about what the header file for such a data structure might look like. In particular, think about how much you tell the client about the structure and what procedure you provide.

Are you done? Here’s my approach.

```
#ifndef _STACK_H_
#define _STACK_H_

/**
 * stack.h
 *   Declarations for a simple implementation of stacks.
 *
 * <insert MIT license>
 */

// +-------+-----------------------------------------------------
// | Types |
// +-------+

typedef struct Stack Stack;

// +------------+----------------------------------------------------
// | Procedures |
// +------------+

/**
```

```
 * Create a new stack.
 */
Stack * stack_new (void);

/**
 * Free the space allocated to a stack.
 */
void stack_free (Stack *stack);

/**
 * Determine the number of elements in the stack.
 */
int stack_size (Stack *stack);

/**
 * Look at the top element of the stack.  Requires that the
 * stack contains at least one element.
 */
int stack_top (Stack *stack);

/**
 * Remove and return the top element of the stack.  Requires that
 * the stack contains at least one element.
 */
int stack_pop (Stack *stack);

/**
 * Add an element to the stack.
 */
void stack_push (Stack *stack, int val);

#endif // _STACK_H_
```

Was your solution similar? If not, that's okay, we'll give you a chance to explore alternatives a bit later. But we'll assume it's similar. What have we done? We've hidden the details of the stack structure from the reader. They just know that they have to pass around values of type `Stack *`. We have a constructor (`stack_new`), a destructor (`stack_free`), two mutators (`stack_push` and `stack_pop`), and two observers (`size`) and (`top`).

Now it's time to implement these stacks. I'll use the standard linked structure approach [11]. We'll start with the nodes for the linked structure.

```
/**
 * The nodes in our stack.
 */
struct StackNode
  {
    int val;                    // The value at the top of the stack
    struct StackNode *next;     // The rest of the values in the stack.
  };
```

This structure will store the node at the top of the stack and the count of values in the stack. I find it useful to keep track of that count because I often like to query the count, and I'd rather have a constant time operation, rather than one that depends on the size of the stack.

```
/**
 * The stack itself.
 */
struct Stack
  {
    int size;                   // How many elements are in the stack?
    struct StackNode *top;      // What's at the top of the stack?
  };
```

Now, we're ready to write the six basic stack operations. You should think about how you might implement those for a minute or two before you see how I've done so [12].

```
Stack *
stack_new (void)
{
  struct Stack *new_stack = (struct Stack *) malloc (sizeof (struct Stack));
  new_stack->size = 0;
  new_stack->top = NULL;
  return new_stack;
} // stack_new

void
stack_free (Stack *stack)
{
  // Free all the nodes in the stack
  struct StackNode *tmp;
  while ((tmp = stack->top) != NULL)
    {
      stack->top = tmp->next;
      free (tmp);
    } // while
  // Free the stack itself
  free (stack);
} // stack_free


int
stack_size (Stack *stack)
{
  return stack->size;
} // stack_size

int
stack_top (Stack *stack)
{
  return stack->top->val;
} // stack_top
```

```
int
stack_pop (Stack *stack)
{
  // Remember the top node and its value
  struct StackNode *tmp = stack->top;
  int top = tmp->val;
  // Drop that node
  stack->top = stack->top->next;
  free (tmp);
  --stack->size;
  // And return the saved value
  return top;
} // stack_pop

void
stack_push (Stack *stack, int val)
{
  struct StackNode *new_top =
    (struct StackNode *) malloc (sizeof (struct StackNode));
  new_top->val = val;
  new_top->next = stack->top->next;
  stack->top = new_top;
} // stack_push
```

As I said, it's not that complicated.

Now, suppose we decided that we wanted to deal with double values rather than integers. What would you do? You could change all the instances of `int` to `double`. Some of those instances relate to the size, which should remain an `int`, so you really have to look carefully at each one.

In addition, if you planned to have both stacks of ints and stack of doubles in the same program, you'd also have to change the name of the various procedures. We should be able to write a regular expression to do that. Think about it for a minute [14]. Done? Here's what I'd write as a substitution command in vi: `:1,$s/^\([a-z_]*\)(/d_\1 (/` [15].

Now we have two nearly identical files. But what happens if we decide to make a change? For example, suppose we decide that instead of freeing the individual nodes in `stack_free`, we should just pop items until the stack is empty.

```
  while (stack->top != NULL)
    stack_pop (stack);
  free (stack);
```

You might also consider what precondition checking to add. In any case, we now have two files to keep track of. Past history suggests that they will become unsynchronized. There is, after all, a motivation for the recommendation to write DRY code.

Can we DRY out this code? Sure. It turns out that macros can help. How? We can write somewhat generic code and use the the text substitution capabilities of the C preprocessor to turn that generic code into type-specific code.

Let's start by rewriting the header file so that the type of stored values are represented by `TYPE` and the type-specific procedure and struct names are nested in a call to `TYPED`. We'll see why in a minute.

```
/**
 * generic-stack.h
 *   Declarations for a simple generic implementation of stacks.
 *
 * <insert MIT license>
 */

#ifdef TYPE

// +--------+------------------------------------------------------
// | Macros |
// +--------+

/**
 * By default, we do not rename procedures and structs.
 */
#ifndef TYPED
#define TYPED(THING) THING
#endif

// +-------+-------------------------------------------------------
// | Types |
// +-------+

typedef struct TYPED(Stack) TYPED(Stack);

// +------------+----------------------------------------------------
// | Procedures |
// +------------+

/**
 * Create a new stack.
 */
void TYPED(stack_new) (void);

/**
 * Free the space allocated to a stack.
 */
void TYPED(stack_free) (TYPED(Stack) *stack);

/**
 * Determine the number of elements in the stack.
 */
int TYPED(stack_size) (TYPED(Stack) *stack);

/**
 * Look at the top element of the stack.  Requires that the
 * stack have at least one element.
```

```
 */
TYPE TYPED(stack_top) (TYPED(Stack) *stack);

/**
 * Remove and return the top element of the stack.  Requires that
 * the stack have at least one element.
 */
TYPE TYPED(stack_pop) (TYPED(Stack) *stack);

/**
 * Add an element to the stack.
 */
void TYPED(stack_push) (TYPED(Stack) *stack, TYPE val);

#endif // TYPE
```

A bit silly looking, isn't it? Well, let's see why it may be useful. First, we'll define `TYPE` as `int`.

```
$ cc -E -DTYPE=int generic-stack.h
# 1 "generic-stack.h"
# 1 ""
# 1 ""
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "" 2
# 1 "generic-stack.h"
# 25 "generic-stack.h"
typedef struct Stack Stack;
# 34 "generic-stack.h"
void stack_new (void);




void stack_free (Stack *stack);




int stack_size (Stack *stack);




int stack_top (Stack *stack);




int stack_pop (Stack *stack);
```

```
void stack_push (Stack *stack, int val);
```

Not that impressive, is it? We're back to our first set of declarations [16], the ones we wrote before we thought about making our code DRY. But let's try a different type for `TYPE`.

```
$ cc -E -DTYPE=double generic-stack.h
# 1 "generic-stack.h"
# 1 ""
# 1 ""
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "" 2
# 1 "generic-stack.h"
# 25 "generic-stack.h"
typedef struct Stack Stack;
# 34 "generic-stack.h"
void stack_new (void);



void stack_free (Stack *stack);



int stack_size (Stack *stack);



double stack_top (Stack *stack);



double stack_pop (Stack *stack);



void stack_push (Stack *stack, double val);
```

Okay, that's nicer. A different compile flag gives us declarations for a different type of stack. What about the `TYPED`? Well, that's a way for us to put a prefix on the various names. We're going to define a macro, named `TYPED`, that adds a prefix. In gcc, we can use the `##` operation to concatenate text. Let's try. I find it easier to

write macros in files, so I'm going to put the definitions in a file, rather than on the command line.

```
#ifndef _DOUBLE_STACK_H_
#define _DOUBLE_STACK_H_

/**
 * double-stack.h
 *   Stacks of doubles.
 *
 * <insert MIT license>
 */

#undef TYPE
#undef TYPED

#define TYPE double
#define TYPED(THING) d_ ## THING

#include "generic-stack.h"

#endif // _DOUBLE_STACK_H_
```

Now, what happens if we preprocess this file?

```
$ cc -E double-stack.h
# 1 "double-stack.h"
# 1 ""
# 1 ""
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 1 "" 2
# 1 "double-stack.h"
# 17 "double-stack.h"
# 1 "generic-stack.h" 1
# 25 "generic-stack.h"
typedef struct d_Stack d_Stack;
# 34 "generic-stack.h"
void d_stack_new (void);




void d_stack_free (d_Stack *stack);




int d_stack_size (d_Stack *stack);
```

```
double d_stack_top (d_Stack *stack);




double d_stack_pop (d_Stack *stack);




void d_stack_push (d_Stack *stack, double val);
# 18 "double-stack.h" 2
```

We have now customized the name as well as the return type of each operation and of the type. If we're careful, we can have `d_Stack` structs store doubles and `i_Stack` structs store ints, and so on and so forth.

Our definition of `double-stack.c` is even more straightforward: Because the macros are defined in this header and we include the header, we don't need to define them again.

```
/**
 * double-stack.c
 *   A simple implementation of stacks of doubles.
 *
 * <insert MIT license>
 */

#include "double-stack.h"
#include "generic-stack.c"
```

As the preprocessor shows, we get the code we want.

```
$ cc -E double-stack.c
...
# 14 "generic-stack.c" 2
# 22 "generic-stack.c"
struct d_StackNode
  {
    double val;
    struct d_StackNode *next;
  };




struct d_Stack
  {
    int size;
    struct d_StackNode *top;
  };
```

```
void
d_stack_new (void)
{
  struct d_Stack *new_stack = (struct d_Stack *) malloc (sizeof (struct d_Stack));
  new_stack->size = 0;
  new_stack->top = ((void *)0);
}


void
d_stack_free (d_Stack *stack)
{

  struct d_StackNode *tmp;
  while ((tmp = stack->top) != ((void *)0))
    {
      stack->top = tmp->next;
      free (tmp);
    }

  free (stack);
}

int
d_stack_size (d_Stack *stack)
{
  return stack->size;
}

double
d_stack_top (d_Stack *stack)
{
  return stack->top->val;
}

double
d_stack_pop (d_Stack *stack)
{

  struct d_StackNode *tmp = stack->top;
  double top = tmp->val;

  stack->top = stack->top->next;
  free (tmp);
  --stack->size;
```

```
  return top;
}

void
d_stack_push (d_Stack *stack, double val)
{
  struct d_StackNode *new_top =
    (struct d_StackNode *) malloc (sizeof (struct d_StackNode));
  new_top->val = val;
  new_top->next = stack->top->next;
  stack->top = new_top;
}
# 9 "double-stack.c" 2
```

We'll leave it as an exercise for the reader to think about other uses of this technique.

At the start of this essay, I mentioned that I saw multiple inclusions of the same file. In case you couldn't tell, these examples of generics show why you might do so.

```
// Use stacks of ints
#undef TYPE
#undef TYPED
#define TYPE int
#define TYPED(THING) i_ ## THING
#include "generic-stack.h"
// Use stacks of doubles
#undef TYPE
#undef TYPED
#define TYPE double
#define TYPED(THING) d_ ## THING
#include "generic-stack.h"
// Use stacks of strings
#undef TYPE
#undef TYPED
#define TYPE char *
#define TYPED(THING) str_ ## THING
#include "generic-stack.h"
```

You should be able to figure out the rest without too much effort. That's about as DRY as I can get data structure code in C.

A colleague writes

> A pointer to production quality implementations wouldn't go amiss in your post. The BSD `sys/queue.h` and `sys/tree.h` list/stack/queue and tree implementations run on a few billion devices. `uthash.h` is newer but might be even more popular in new code.

> A cute related trick I saw recently was using macros to generate specialized instances of C sorting functions implementing the posix qsort() interface. They were called from the generic version in the (likely) case that the caller was sorting an array of primitive types.

That note reminds me that I'm definitely not a working professional C programmer, since I didn't know about any of this. *nix users can learn more about `sys/queue.h` by typing `man 3 queue`. It looks like uthash is an independent project. The user guide (http://troydhanson.github.io/uthash/userguide.html) is quite informative. I'm sure that my students will appreciate the associated assignments.

---

[1] Sorry, I have no idea what it was.

[2] Okay, good programmers use data structures from a library *and* know how to build the data structures if the library does not exist.

[3] Both Java and Scheme let you create homogeneous [4] and heterogeneous [9] data structures. Java will provide some automatic type checking on the homogeneous data structures, while Scheme will generally force you to do the type checking yourself.

[4] There are at least two pronunciations of "homogeneous" that I've heard. One is "ho-modge-en-us" [5]; the other is "ho-mo-gee-nee-us". You say puh-tay-toe, I say puh-tah-toe [6].

[5] I should really learn the international phonetic alphabet.

[6] In either case, Dan Quayle ends it with an "e" [7].

[7] I should not make fun of Dan Quayle. I'm told that he was actually intelligent [8].

[8] He's still alive, so he is probably still intelligent.

[9] Two choices here, too. Some say "het-er-oh-gee-nee-us"; others say "het-er-odge-en-us". If I'm not mistaken, the primary pronunciation of homogeneous is with the "modge-en-us", but the primary pronunciation of "heterogeneous" is with the "gee-nee-us".

[10] There is a reason that many programmers work in garbage collected languages.

[11] Programmers generally implement stacks with arrays or as linked structures. We will not get into the relative strengths and weaknesses of each approach.

[12] That's a good interview question. "*Implement linked stacks. You have ten minutes.*"

[14] That's a harder interview question.

[15] There's a reason regular expressions are sometimes considered "write only".

[16] Well, the main code in the original declarations. We've lost the comments because we're preprocessing.

Version 1.1 of 2017-04-13.

The opinions stated herein are those of Samuel A. Rebelsky and do not necessarily reflect those of Grinnell College, Grinnell's Computer Science Department, the Rebelsky family, CMD-IT, SIGCAS, SIGCSE, any other organizations I am or have been affiliated with, or even most other sentient beings.

Check accessibility with WAVE (http://wave.webaim.org/refer).

Copyright © 2016–25 Samuel A. Rebelsky.

 (http://creativecommons.org/licenses/by/4.0/)

“SamR's Assorted Musings and Rants: Using macros for generic structures in C” by Samuel A. Rebelsky is licensed under a Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/).

This Web site was built using Markdown, some custom scripts, Twitter Bootstrap (http://getbootstrap.com), and the Bootswatch Readable Theme (https://bootswatch.com/3/readable/).